



International Islamic University Chittagong

Department of Computer Science and Engineering

Lab Report

Lab No 3: Implementation of Midpoint Circle Algorithm

Course Title: Computer Graphics Lab

Course Code: CSE-4742

Session: Autumn 2026

Semester: 7th

Section: 7BF

Student Name	Tahasina Tasnim Afra
Student ID	C233456
Department	Computer Science and Engineering

Submitted To

Ms. Sultana Tasnim Jahan
Lecturer, Department of CSE, IIUC

Submission Date:

September 12, 2026

1. Number & Name of the Experiment

Lab 3: Implementation of the Midpoint Circle Algorithm

2. Introduction

The Midpoint Circle Algorithm is an incremental circle scan-conversion algorithm used in computer graphics to generate a circle using integer pixel coordinates. Instead of calculating the complete circle, it determines the pixels for one portion and uses the circle's symmetry to generate the remaining points.

For a circle with center (h, k) and radius r , the circle is defined by

$$(x - h)^2 + (y - k)^2 = r^2.$$

For a circle centered at $(0, 0)$, the equation becomes

$$x^2 + y^2 = r^2.$$

The algorithm starts at the top of the circle with

$$x = 0, \quad y = r$$

and uses the initial decision parameter

$$p = 1 - r.$$

At each step, the decision parameter determines which pixel should be selected next. The algorithm mainly uses integer addition, subtraction, and comparison, making it efficient for raster displays.

3. Midpoint Circle Algorithm

The Midpoint Circle Algorithm generates the pixels of a circle incrementally using a decision parameter. For a circle centered at $(0, 0)$ with radius r , the algorithm starts at the top of the circle and calculates the points from 90° to 45° . The remaining points are obtained using eight-way symmetry.

The initial values are

$$x = 0, \quad y = r, \quad p = 1 - r.$$

The algorithm continues while

$$x \leq y.$$

Initial Point

The algorithm starts from the top of the circle:

$$(x, y) = (0, r)$$

For example, if $r = 200$, the initial point is

$$(0, 200).$$

The initial decision parameter is

$$p = 1 - r.$$

For $r = 200$,

$$p = 1 - 200 = -199.$$

Decision Parameter

At each iteration, the value of p determines the next pixel. There are two cases.

Case 1: $p < 0$

When

$$p < 0,$$

the midpoint is inside the circle, so y is not decreased. The decision parameter is updated as

$$p = p + 2x + 3.$$

Then x is increased by one:

$$x = x + 1.$$

Case 2: $p \geq 0$

When

$$p \geq 0,$$

the midpoint is outside or on the circle. Therefore, x is increased and y is decreased. The decision parameter is updated as

$$p = p + 2(x - y) + 5$$

and

$$y = y - 1.$$

After this, x is increased by one:

$$x = x + 1.$$

Eight-Way Symmetry

A circle is symmetric about the horizontal and vertical axes and the two diagonal lines. Therefore, when one point (x, y) is calculated, seven additional points can be obtained by symmetry.

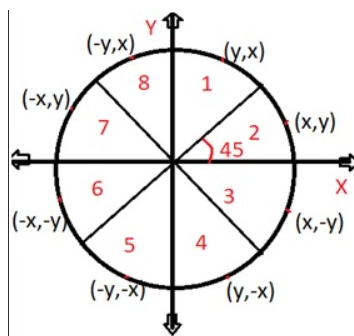


Figure 1: Eight-way symmetry of a circle

For a circle centered at (h, k) , the eight symmetric points are

$$\begin{array}{ll} (h+x, k+y), & (h+y, k+x), \\ (h-y, k+x), & (h-x, k+y), \\ (h-x, k-y), & (h-y, k-x), \\ (h+y, k-x), & (h+x, k-y). \end{array}$$

In the program, the mathematical center $(0, 0)$ is mapped to the center of the graphics window. The screen y -coordinate increases downward, so the y -coordinate is adjusted accordingly.

The points are plotted using `putpixel()`.

Stopping Condition

The algorithm calculates points only up to the 45° boundary. The loop continues while

$$\boxed{x \leq y}.$$

When $x > y$, the algorithm stops because the remaining points are already obtained through eight-way symmetry.

At approximately the 45° position,

$$x \approx y \approx \frac{r}{\sqrt{2}}.$$

For $r = 200$, the calculated points end at approximately $(141, 142)$.

Implementation

The implementation follows the incremental decision-parameter method. The initial values are

```
x = 0;
y = r;
p = 1 - r;
```

The algorithm then repeatedly plots the current point and its seven symmetric points while

$$x \leq y.$$

If $p < 0$, the decision parameter is updated using

$$p = p + 2x + 3.$$

Otherwise, it is updated using

$$p = p + 2(x - y) + 5$$

and y is decreased by one. In both cases, x is then increased by one.

The main steps are:

1. Set $x = 0$, $y = r$, and $p = 1 - r$.
2. Check whether $x \leq y$.
3. Plot the current point and its seven symmetric points.
4. If $p < 0$, update p using $p = p + 2x + 3$.
5. Otherwise, update p using $p = p + 2(x - y) + 5$ and decrease y by one.
6. Increase x by one.
7. Repeat until $x > y$.

In the C++ program, the `MidpointCircle()` function performs the main algorithm, while `plotCirclePoints()` generates the eight symmetric points using `putpixel()`.

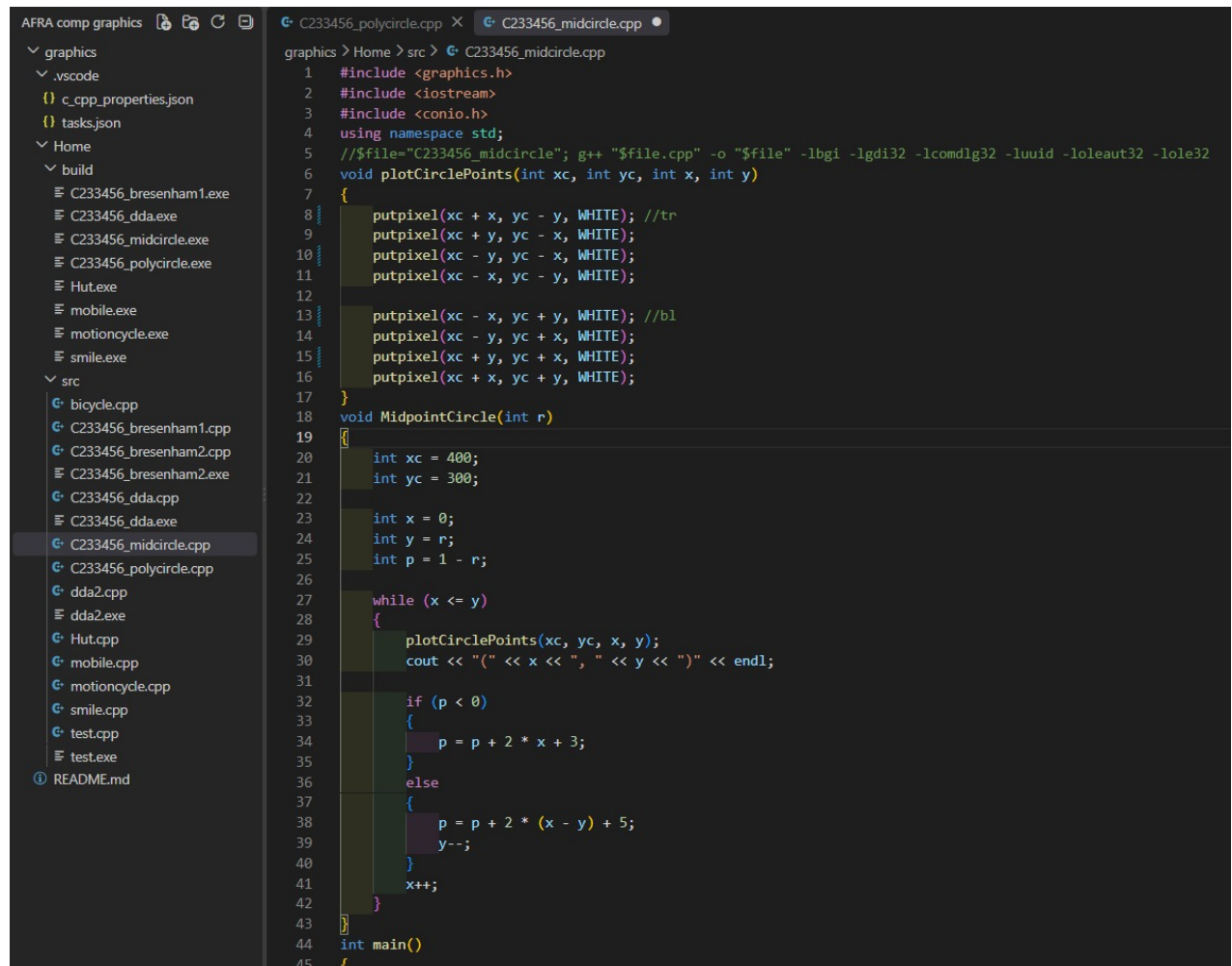
The program also displays the calculated (x, y) coordinates using `cout`. A small `delay()` is included to observe the incremental drawing process.

4. Program with Screenshot

The Midpoint Circle Algorithm was implemented in C++ using the `graphics.h` graphics library. The program uses the decision parameter to determine the next pixel and applies eight-way symmetry to generate the complete circle. The code can be found in the following GitHub repository: <https://github.com/afrahackerman/AFRA-comp-graphics>

Code Screenshot

The following screenshot shows the complete implementation of the Midpoint Circle Algorithm:



```
1  #include <graphics.h>
2  #include <iostream>
3  #include <conio.h>
4  using namespace std;
5  // $file="C233456_midcircle"; g++ "$file.cpp" -o "$file" -lbgf -lstdc++ -lcomdlg32 -luuid -oleaut32 -ole32
6  void plotCirclePoints(int xc, int yc, int x, int y)
7  {
8      putpixel(xc + x, yc - y, WHITE); //tr
9      putpixel(xc + y, yc - x, WHITE);
10     putpixel(xc - y, yc - x, WHITE);
11     putpixel(xc - x, yc - y, WHITE);
12
13     putpixel(xc - x, yc + y, WHITE); //bl
14     putpixel(xc - y, yc + x, WHITE);
15     putpixel(xc + y, yc + x, WHITE);
16     putpixel(xc + x, yc + y, WHITE);
17 }
18 void MidpointCircle(int r)
19 {
20     int xc = 400;
21     int yc = 300;
22
23     int x = 0;
24     int y = r;
25     int p = 1 - r;
26
27     while (x <= y)
28     {
29         plotCirclePoints(xc, yc, x, y);
30         cout << "(" << x << ", " << y << ")" << endl;
31
32         if (p < 0)
33         {
34             p = p + 2 * x + 3;
35         }
36         else
37         {
38             p = p + 2 * (x - y) + 5;
39             y--;
40         }
41         x++;
42     }
43 }
44 int main()
45 {
```

Figure 2: C++ implementation of the Midpoint Circle Algorithm

5. Output/Result

The program was successfully executed and produced a complete circle using the Midpoint Circle Algorithm. The circle was generated by calculating the pixels for one portion of the circle and using eight-way symmetry to plot the remaining points.

For a radius of 200, the algorithm started from the point (0, 200) and continued until the condition $x \leq y$ was no longer satisfied. The calculated points were plotted using `putpixel()`.

The ID (C233456) was also displayed in the graphics window as required.



Figure 3: Output of the Midpoint Circle Algorithm for radius $r = 200$

6. Discussion

The Midpoint Circle Algorithm successfully generated a circle using incremental calculations and a decision parameter. The algorithm started with $x = 0$, $y = r$, and the initial decision parameter $p = 1 - r$.

At each iteration, the value of the decision parameter was used to determine the next pixel. When $p < 0$, the decision parameter was updated using

$$p = p + 2x + 3.$$

When $p \geq 0$, the algorithm updated the decision parameter using

$$p = p + 2(x - y) + 5$$

and decreased y by one. The value of x was then increased by one in every iteration. Only one portion of the circle was calculated directly. The other points were obtained using eight-way symmetry, which allowed the complete circle to be generated efficiently.

The experiment demonstrates how a mathematical circle can be converted into a set of discrete pixels for display on a raster screen.

For the selected radius of 200, the calculated points began at $(0, 200)$ and continued until approximately the 45° position, ending at $(141, 142)$. The remaining points were generated through symmetry.

7. Conclusion

The Midpoint Circle Algorithm was successfully implemented in C++ using the `graphics.h` graphics library. The experiment demonstrated how a circle can be scan converted into pixels using an incremental decision parameter rather than repeatedly calculating the circle equation.

The algorithm starts from $(0, r)$, determines the appropriate pixels using the decision parameter, and uses eight-way symmetry to produce the complete circle. The experiment provided an understanding of circle rasterization and how individual pixels can be selected and displayed using `putpixel()`.